

一、BFS

二、List 相关操作： 定义 struct 节点，新建立节点（new，delete）

三、快慢指针判断有无环

```
#include "Node.h"
#include<iostream>
using namespace std;
bool check(node *head){
    //处理空链表或只有一个节点且无环的情况
    if(head==NULL||head->next==NULL)
        return false;

    //初始化快慢指针
    node* slow=head;
    node* fast=head->next;

    //遍历列表，快慢指针相遇即成环
    while(slow!=fast){
        //快指针到末尾，无环
        if(fast==NULL||fast->next==NULL)
            return false;

        slow=slow->next;//移动一步
        fast=fast->next->next;//移动两部（更快）
    }
    return true;
}
```

四、二分查找： 求 target 最右出现

1.循环条件<=的 left 和 right 的变化（在循环内得到答案）

```
#include "binSearch.h"
int binSearch(const int s[], const int size, const int target)
{
    int left=0,right=size-1,result=-1;;
    int mid;

    while(left<=right){
        mid=(left+right)/2;

        if(s[mid]==target){
            result=mid;
            left=mid+1;
        }else if(s[mid]>target){
            right=mid-1;
        }else{
            left=mid+1;
        }
    }

    return result;
}
```

五、sort

1.QUICKSORT 的 HOARE'S 划分

```

void sort_nums(vector<int>& nums,int left,int right){
    if(left>=right)//递归中止条件
        return;

    int x,y;//partition过程
    int mid = left + (right - left) / 2;//用中点做pivot值
    swap(nums[mid],nums[left]);

    int pivot=nums[left];//pivot值移动到最前便于未排序部分移动
    x=left+1;
    y=right;

    while(x<=y){//确保所有元素都被比较
        while(nums[x]<=pivot&&x<=y)x++;//指针移动条件和循环终止条件
        统一
        while(nums[y]>=pivot&&x<=y)y--;

        if (x < y) {
            swap(nums[x], nums[y]);//标准库swap函数简化
        }
    }
    swap(nums[left], nums[y]);//y是pivot应在位置

    sort_nums(nums, left, y - 1);//递归快速排序
    sort_nums(nums, y + 1, right);
}

```

2.MERGESORT（注意区间边界是否可取）★

```

void inverse_number(int* A,int x,int y,int*T){
    if(y-x>1){//闭区间，区间元素小于等于一时递归结束
        int mid=x+(y-x)/2;
        int a=x,b=mid,c=x;
        inverse_number(A,x,mid,T);//分别对[x,m)以及[m,y)进行递归
        inverse_number(A,mid,y,T);
        while(a<mid||b<y){//归并过程
            if(b>=y||(a<mid&&A[a]<=A[b])){//左子数组元素更小，直接
                放入T，无逆序对
                T[c++]=A[a++];
            }else{//右子数组元素更小，放入T，并且计算逆序对数量
                T[c++]=A[b++];
                sum+=mid-a;//[a,mid)间元素都比右子数组元素大，都能组
                成逆序对
            }
        }
        for(int i=x;i<y;i++){//保证后续递归是有序数组
            A[i]=T[i];
        }
    }
}

```

指针版本（注意一边为空情况，走到头）

```

void mergesort(linkedlist *&head, int len){

    if (len <= 1 || head == nullptr) { //递归终止条件
        return;
    }

    linkedlist *second=head;
    linkedlist *first=head;
    linkedlist *prev=nullptr;
    int mid=len/2;

    for(int i=0;i<mid;i++){
        prev=second;
        second=second->next;
    }
    prev->next=nullptr; //分割两个子链表

    mergesort(first,mid);
    mergesort(second,len-mid); //注意分割的两个子链表的长度

    linkedlist* new_head=nullptr;
    linkedlist* last=nullptr; // 合并过程中当前链表的尾

    //初始化新链表的头结点
    if (first->data <= second->data) {
        new_head= first;
        last = first;
        first = first->next;
    } else {
        new_head = second;
        last = second;
        second = second->next;
    }

    // 循环合并两个子链表的剩余节点，使新的链表有序
    while(first!=nullptr&&second!=nullptr){
        if(first->data<=second->data){
            last->next=first;
            last=first;
            first=first->next;
        }else{
            last->next=second;
            last=second;
            second=second->next;
        }
    }

    if(first==nullptr) // 处理剩余节点（其中一个子链表已遍历完）
        last->next=second;
    else
        last->next=first;

    head=new_head; //将新头指针赋值给原头指针
}

```

六、树的相关操作：注意根节点 0/1

1. 二叉搜索树的构造

```
//向二叉搜索树（BST）中插入结点
b_node* insert_node(b_node* root,int data){
    b_node* new_node=new b_node(data);

    if(root==nullptr)return new_node;//孔数时直接作为根节点，
    注意特殊性

    b_node* curr=root;
    b_node* parent=nullptr;

    //找到插入位置
    while(curr!=nullptr){
        parent=curr;

        if(data<curr->data){
            curr=curr->left;
        }else{
            curr=curr->right;
        }
    }

    //根据大小比较插入父节点的左子树or右子树
    if(data<parent->data){
        parent->left=new_node;
    }else{
        parent->right=new_node;
    }
    return root;
}
```

2. 节点定义

```
//结点结构
struct b_node{
    int data;
    b_node* left;
    b_node* right;
    b_node(int x):data(x),left(nullptr),right(nullptr){}
};
```

3. 清除树的内存

```
//释放树的内存，new与delete搭配
void deletetree(b_node* root){
    if(root==nullptr)return;
    deletetree(root->left);
    deletetree(root->right);
    delete root;
}
```

4.求树高 (Z23 判断是否平衡) ★★

```
//递归得到结点的高度
int find_h(const Node* root){
    if(root==nullptr)return 0;

    int left_h=find_h(root->lc);
    int right_h=find_h(root->rc);

    return 1+max(left_h,right_h); //取最大值才是结点高度，注意
    //一个结点高度为1
}
```

5.已知前序和中序求后序遍历顺序: Z27

6.Z22 交错路径: 学习类的封装功能使用

★★★7.并查集的使用: 传递性, 找亲戚 (需自己实现)

main.cpp

```
1  #include<iostream>
2  using namespace std;
3  #include<vector>
4
5  int find(int x,vector<int>& parent){
6      return (parent[x]==-1)?x:find(parent[x],parent);
7  }
8
9  int main(){
10     vector<int> parent(1000,-1);
11     int x,y;
12     while(cin>>x>>y){
13         int root_x=find(x,parent);
14         int root_y=find(y,parent);
15
16         if(root_x!=root_y){
17             cout<<x<<' '<<y<<endl;
18             parent[root_y]=root_x; //union(root_y,root_x);
19         }
20     }
21     return 0;
22 }
```

7.多叉树转二叉树: 注意 HASH 表的使用

```

node* build_BT(vector<int>& num, int n){
    if(n == 0) return nullptr;

    // 哈希表: 存储节点值到节点指针的映射, 避免重复查找
    unordered_map<int, node*> node_map;

    // 创建根节点
    node* root = new node(1);
    node_map[1] = root;

    for(int i = 1; i < n; ++i){
        int node_val = i + 1;
        int parent_val = num[i];
        node* new_node = new node(node_val);
        node_map[node_val] = new_node;

        // 从哈希表直接获取父节点 (O(1)时间)
        auto it = node_map.find(parent_val);
        if(it == node_map.end()) continue;
        node* parent = it->second;

        // 添加新节点
        if(parent->fchild == nullptr){
            parent->fchild = new_node;
        } else {
            // 找到父节点当前最后一个孩子
            node* last_sibling = parent->fchild;
            while(last_sibling->nsibling != nullptr){
                last_sibling = last_sibling->nsibling;
            }
            last_sibling->nsibling = new_node;
        }
    }
    return root;
}

```

8. 树的 BFS

```

void traverse(node* root){
    if (root == nullptr) return;
    vector<vector<int>> result;

    queue<node*> q;
    q.push(root);

    while (!q.empty()) {
        int level_size = q.size();
        vector<int> currentLevel;//记录本层节点

        for (int i = 0; i < level_size; ++i) {
            node* cur = q.front();
            q.pop();

            currentLevel.push_back(cur->data);

            //子节点入队
            if(cur->fchild!=nullptr){
                q.push(cur->fchild);
            }

            if(cur->nsibling!=nullptr){
                q.push(cur->nsibling);
            }
        }
        result.push_back(currentLevel);
    }

    for(auto& level:result){
        for(int data:level){
            cout<<data<<' ';
        }
    }
}

```

9.HUFFMAN 树：学习优先队列的使用，记得跳过空结点

需要使用优先队列，注意 把内部结点加入队列中，还要注意循环终止条件（size=1 来赋值 root 结点），合并过程只需要进行 s-1 次

```

//自定义优先队列比较函数
struct Compare{
    bool operator()(node* a,node* b){
        return a->val>b->val;
    }
};

//构建huffman tree
node* build_tree(priority_queue<node*,vector<node*>,
Compare>& q){
    while(q.size()>=2){
        node* fir=q.top();
        q.pop();
        node* sec=q.top();
        q.pop();

        node* parent=new node(-1,fir->val+sec->val);
        parent->left=fir;
        parent->right=sec;

        q.push(parent);
    }

    node*root=q.top();
    return root;
}

```

七、学习多种特殊输入：如中间有换行符

注：有时候需要吸收输入框中的废物

1.cin.get()

2.cin.getline()

3.getline(cin,line)

4.关闭同步，加快输入速度

ios::sync_with_stdio(false)

Cin.tie(nullptr)

5.scanf

6.跳过换行符

Cin.ignore()

7.使用标记

```

while (getline(cin, line)) {
    if (line.empty()) {
        // 遇到空行，切换到查询阶段
        inDict = false;
        continue;
    }

    istringstream iss(line);
    if (inDict) {
        // 词典阶段：读取两个单词
        if (iss >> En >> Foreign) {
            dict[Foreign] = En;
        }
    }
}

```

输入描述

There are multiple input cases. For one case, first are two lines of two integers N and M , ($N \leq 200, M \leq N * N/2$), that means the number of cities and the number of roads. And Merlin stands at city $N - 1$. After that, there are M lines. Each line contains two integers i and j , what means that there is a road from city i to city j . The input is terminated by $N = 0$.

.....

```

25     }
26 }
27 cout<<"I can't post the letter"<<
28 }
29
30 int main(){
31     int n,m;
32     while(cin>>n&& n!=0){
33         int m;
34         cin>>m;
35         vector<vector<int>>> edge(n);
36
37         for(int i=0;i<m;i++){
38             int x,v;

```

八、图的相关操作：注意有无向

1. **DFS**: 此题为求连通图数量，学习邻接表的构建
z14 也是 DFS 求进入与出去的时间差

```

void dfs(int u,vector<vector<int>>& graph,vector<bool>&
visited){
    for(int v:graph[u]){
        if(!visited[v])
        {
            visited[v]=true;
            dfs(v,graph,visited);
        }
    }
}

```

```

int main(){
    int n,m;
    cin>>n>>m;

    //构建无向图（邻接表）
    vector<vector<int>> graph(n+1,vector<int>(n+1));
    for(int i=0;i<m;i++){
        int x,y;
        cin>>x>>y;
        graph[x].push_back(y);
        graph[y].push_back(x);
    }

    vector<bool> visited(n+1,false);
    int sum=0;

    for(int i=1;i<=n;i++){
        if(!visited[i])
            sum++;
        visited[i]=true;
        dfs(i,graph,visited);
    }
}

```

2.先判断图是否连通：求路径，欧拉回路（针对无向图），注意空图的特殊处理

```

int main() {
    ios::sync_with_stdio(false); // 关闭同步流, 加速输入输出, 提升输入输出效率
    cin.tie(nullptr);

    int N, M;
    while (cin >> N >> M && N != 0) {
        int sig = 1;

        if (M < N - 1) {
            sig = 0;
            // 消耗剩余的输入!!!
            for (int i = 0; i < M; ++i) {
                int x, y;
                cin >> x >> y;
            }
        } else {
            vector<vector<int>> city_path(N); // 正确初始化邻接表

            while (M--) {
                int x, y;
                cin >> x >> y;
                city_path[x].push_back(y);
            }

            is_post(city_path, sig, N);
        }

        // 灵活使用三元运算符
        cout << (sig ? "I can post the letter" : "I can't post the letter") << '\n';
    }

    return 0;
}

```

3. 拓扑排序：此题按字典序，使用 INDEGREE 数组

可用于安排任务/判断是否为有向无环图——仍有入度（类似遍历可以用来判断是否连通）

本题为 LEXICAL ORDER（字典序）

注意 PRIORITY_QUEUE 默认大顶堆，取元素使用 TOP()

```

// 拓扑排序主函数（按字典序输出）
void topological_order(vector<vector<int>>& g, vector<int>&
in_degree, int n) {
    set<int> zero_in; //自动按字典序排序

    // 初始化：将所有入度为0的节点加入set
    for (int i = 1; i <= n; i++) {
        if (in_degree[i] == 0) {
            zero_in.insert(i);
        }
    }

    // 取set首元素处理，直到无入度为0的节点
    while (!zero_in.empty()) {
        int curr = *zero_in.begin();
        zero_in.erase(zero_in.begin()); // 移除已处理的节点
        cout << curr << " ";

        // 遍历当前节点的所有邻接节点，入度-1
        for (int neighbor : g[curr]) {
            in_degree[neighbor]--;
            // 若邻接节点入度变为0，加入set等待处理
            if (in_degree[neighbor] == 0) {
                zero_in.insert(neighbor);
            }
        }
    }
}

```

4.最小生成树： 最大生成森林的 PRIM 算法和 KRUSKAL 算法实现

★邻接表和邻接矩阵的选取——邻接表用 `FOR(INT I=0;I<N;I++)`，注意 `DIS[0]`不可改变，邻接矩阵用 `FOR(AUTO E:EDGE[I])`，在初始化时不会改变 `DIS[0]`

①KRUSKAL

```

struct edge{
    int u,v,w;
};

```

（善于运用结构体）

```

//自定义比较函数，便于连接权值最大的边
bool cmp(const edge n1,const edge n2){
    return n1.w>n2.w;
}

void kruskal(vector<edge>& e,vector<int>& parent,
vector<bool>& isin,int n){
    int total=0;
    sort(e.begin(),e.end(),cmp); //优先选权值大的边

    //初始化并查集
    for(int i=0;i<n;i++){
        parent[i]=i;
    }

    //遍历排序后的边，构建最大生成森林
    for(auto ed:e){
        int u=ed.u,v=ed.v,w=ed.w;
        if(unite(u,v,parent)){ //合并不连通的边
            total+=w;
            isin[u]=true;
            isin[v]=true;
        }
    }

    cout<<total<<endl;
}

```

②PRIM

注意 DIS[START]=0，不可被边覆盖

在单源中算法只需要循环 N-1 次加入剩下的结点

```

if(!isin[i]){
    //初始化连通分量起点
    dis[i]=0;
    isin[i]=true;

    //初始化起始点临界距离，注：最大
    for(auto ed:edge[i]){
        dis[ed.v]=max(dis[ed.v],ed.w);//一定是要更大
        才可以改变权值
    }
    //最多执行n-1次，把除起始点外的结点加入森林！！
    for(int j=1;j<n;j++){
        int u=-1;
        int max_path=INT_MIN;//选取最小

        //找未加入的最大权值点
        for(int k=0;k<n;k++){
            if(!isin[k]&&dis[k]>max_path){
                max_path=dis[k];
                u=k;
            }
        }

        if(u==-1)break;//遍历完未找到点，不再连通

        isin[u]=true;
        total+=dis[u];
        //更新邻接边，注意一定是要比原来更大才更新
        for(auto e:edge[u]){
            int v=e.v,w=e.w;
            if(!isin[v]&&dis[v]<w)
                dis[v]=w;
        }
    }
}

```

5.最短路径： 注意两点之间多条路径的筛选

①双向添加，注意可能两点之间多条路径

②防止溢出，先检测两点间是否有路再 RELAX

```

void Floyd_Warshall(vector<vector<int>>& dis,
vector<vector<int>>& edge,int n){

    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            if(edge[i][j]==10000)continue;
            dis[i][j]=edge[i][j];
        }
    }

    for(int i=1;i<=n;i++){
        dis[i][i]=0;
    }

    for(int k=1;k<=n;k++){
        for(int i=1;i<=n;i++){
            for(int j=1;j<=n;j++){
                if(dis[i][k] != 10000 && dis[k][j] !=
                10000&&dis[i][j]>dis[i][k]+dis[k][j])
                    dis[i][j]=dis[i][k]+dis[k][j];
            }
        }
    }
}

```

九.PRIORITY_QUEUE 的使用:

①默认大顶堆

使用 sort 的算法: `sort(students.begin(), students.end(), compare);`

```
priority_queue<int, vector<int>, greater<int>> pq;
```

②

```
struct node{
    char c;
    int fre;
    node* left;
    node* right;
    node(char c, int fre):c(c), fre(fre), left(nullptr), right(nullptr){}
};

class cmp{
public:
    bool operator()(const node* n1, const node* n2){
        return n1->fre==n2->fre?n1->c>n2->c:n1->fre>n2->fre;
    }
};

node* construct_tree(priority_queue<node*, vector<node*>, cmp>& pq){
    while(pq.size()!=1){
        node* right=pq.top();
        pq.pop();
        node* left=pq.top();
        pq.pop();

        node* cur=new node('Z', left->fre+right->fre);
        cur->left=left;
        cur->right=right;
        pq.push(cur);
    }

    node* root=pq.top();
    return root;
}
```

注: 自定义 cmp 函数来使用优先队列

建树过程 (本题为 huffman 树)

主函数中: `pq.push(new node(cc, ff));`

+.hash

1.平方探测法：注意是在 h（哈希映射）的基础上改变位置

```
for(int i=0;i<n;i++){
    int place=numname[i]%m;//取模对应房号
    int start=1;//平方探测法的i初始值
    int new_place=place;

    while(fang[new_place]!="NULL"){//找到空房
        sum++;
        new_place=(place+start*start)%m;//注意取模！防止溢出
        start++;//i++
    }
    fang[new_place]=names[i];
    sum++;
}
```

2.线性探测

```
void find_house(int num,string name,vector<string>& house,
int m,float& sum){
    int pos=num%m;
    sum+=1.0;

    while(house[pos]!="NULL"){
        pos=(pos+1)%m;
        sum++;
    }

    house[pos]=name;
}
```

十一、堆

建堆（插入法）

输出堆顶元素

```

void heap::push(int x) {
    h[++n]=x;

    int i=n;
    while(i>1){
        int father=i/2;
        if(h[father]>h[i]){
            swap(h[father],h[i]);
            i=father;
        }else{
            break;
        }
    }
}

void heap::pop() {
    swap(h[n],h[1]);
    n--;

    int p=1;
    while(2*p<=n){
        int left=2*p;
        int swap_n=p;
        if(h[left]<h[p]){
            swap_n=left;
        }
        if(left+1<=n&&h[left+1]<h[swap_n])swap_n=left+1;

        if(swap_n!=p){
            swap(h[swap_n],h[p]);
            p=swap_n;
        }else{
            break;
        }
    }
}

```

十二、字符串匹配 KMP 算法

```
vector<int> getNext(const string& P) {
    int m = P.size();
    vector<int> next(m, 0);
    int j = 0;
    for(int i = 1; i < m; ++i) {
        while(j > 0 && P[i] != P[j]) {
            j = next[j - 1];
        }
        if(P[i] == P[j]) {
            j++;
        }
        next[i] = j;
    }
    return next;
}

int KMP(const string& S, const string& P, const vector<int>& next) {
    int n = S.size();
    int m = P.size();
    if(m == 0) return 0;
    int j = 0;
    for(int i = 0; i < n; ++i) {
        while(j > 0 && S[i] != P[j]) {
            j = next[j - 1];
        }
        if(S[i] == P[j]) {
            j++;
        }
        if(j == m) {
            return i - m + 1;
        }
    }
    return -1;
}
```